

INSTITUTO DE EDUCACIÓN SECUNDARIA JOSÉ PLANES

**Departamento de Informática y Comunicaciones
Técnico Superior en Desarrollo de aplicaciones Web**

C/ Maestro Pérez Abadía, 2
30100 Espinardo – Murcia
T. 968 834 605
30010577@murciaeduca.es
www.iesjoseplanes.es

Memoria del proyecto

Desarrollo de aplicaciones web

OasisClub

Autores/as:

Emilio Benítez Lozano

Álvaro Franco García-Servet

Profesor/a-coordinador/a:

Fernando Domingo Ruiz

Murcia, Junio de 2026



Esta obra está bajo una [licencia de Creative Commons Reconocimiento-CompartirIgual 4.0 Internacional](https://creativecommons.org/licenses/by-sa/4.0/).

Contenido

1. Resumen extendido
 2. Palabras clave
 3. Introducción
 4. Estado del arte/trabajos relacionados
 5. Análisis de objetivos y metodología
 6. Diseño y resolución del trabajo realizado
 7. Presupuesto
 8. Conclusiones y vías futuras
 9. Bibliografía/Webgrafía
-

1. Resumen extendido

OasisClub es una aplicación web completa para la gestión de un club deportivo, que permite a los socios reservar pistas de pádel y fútbol, inscribirse en eventos, consultar sus rutinas de gimnasio y gestionar su perfil. Por otro lado, los administradores pueden gestionar clientes, pistas, reservas y eventos desde un panel dedicado. La aplicación incluye un sistema de pagos integrado con Stripe, envío de correos electrónicos automáticos, y un sistema de seguridad basado en JWT con control de acceso por roles.

Para el desarrollo hemos utilizado tecnologías modernas y actuales: Angular 21 con componentes standalone y Signals en el frontend, Spring Boot 4 con Java 21 en el backend, PostgreSQL como base de datos, y Docker para el entorno de desarrollo. El despliegue se ha hecho en Render para el backend y Vercel para el frontend, consiguiendo tener la aplicación accesible desde cualquier parte. Aunque el proyecto nació como una propuesta real para un gimnasio de nuestra zona, el cliente finalmente no aceptó el encargo, así que lo transformamos en un proyecto propio donde pudimos aplicar todo lo que habíamos aprendido sin restricciones externas.

2. Palabras clave

club deportivo, reservas online, pistas pádel, pistas fútbol, gestión deportiva, Angular, Spring Boot, Java, PostgreSQL, Stripe, pagos online, JWT, seguridad web, API REST, responsive, eventos deportivos, gimnasio, rutinas, Docker, despliegue cloud

3. Introducción

En primer lugar, vimos que el problema que tenía el gimnasio era la gestión de las reservas, ya que le ocupaba mucho tiempo al hacerlo manualmente en una hoja de Excel. Entonces nos pusimos a pensar y a investigar cuál sería la mejor solución para este problema.

Investigando vimos que había páginas y aplicaciones que gestionaban las reservas online, pero realmente eran muy caras para lo que el cliente podía soportar, entonces dimos el paso de replicar el funcionamiento de esas apps, ya que el gimnasio tampoco tenía una página web propia, así que podíamos solucionar dos cosas a la vez, la falta de la web y la gestión de las reservas. Nos pusimos manos a la obra, y al final acabamos haciéndole una web a medida, que le facilita el trabajo, y con la que tiene una imagen y un sitio donde tener toda la información que la gente busca sobre el gimnasio.

Cuando hicimos una maqueta de cómo podría quedar el proyecto se lo presentamos al dueño del gimnasio, este nos dijo que tenía que pensárselo ya que había hecho recientemente una inversión muy grande. Al final lo rechazó debido a que tenía que dar el paso, y hacerlo implicaba cambiar tanto su forma de trabajar, como el trato con los clientes. Por lo tanto decidimos hacer la página a nuestro gusto, y hacerla como si fuera de un gimnasio nuestro. De ahí nace la idea de OasisClub.

Una vez que decidimos que el proyecto iba a ser nuestro, nos sentamos a definir bien qué queríamos hacer. Teníamos claro que no queríamos una web estática tipo folleto, sino algo funcional de verdad, con un sistema de reservas real que funcionara con pagos, con un panel de administración completo, y con funcionalidades que fueran más allá de lo típico. Queríamos que si alguien entraba a la web, pudiera reservar una pista, pagar, recibir un correo de confirmación, ver su rutina de gimnasio, apuntarse a eventos... todo sin tener que llamar por teléfono ni enviar un WhatsApp.

El reparto del trabajo lo hicimos de forma bastante natural: Álvaro se encargó del backend y yo (Emilio) del frontend. Obviamente hubo cosas que hicimos juntos, como definir los contratos de la API, los DTOs, las pruebas integradas entre frontend y backend, y todo el tema de despliegue. Pero en general cada uno tiró por su lado y nos íbamos juntando para integrar y probar.

4. Estado del arte/trabajos relacionados

Antes de ponernos a programar, estuvimos investigando qué había ya en el mercado para gestión de clubs deportivos y reservas de pistas. Encontramos bastantes opciones, pero todas tenían algún problema para nuestro caso.

Playtomic (<https://playtomic.io>) es probablemente la más conocida para reservas de pádel. Está muy bien, tiene una app muy pulida y funciona genial para el usuario final. Pero el problema es que Playtomic cobra una comisión por cada reserva, y para un club pequeño eso se come los márgenes. Además, al usar Playtomic estás metiendo a tus clientes en su plataforma, donde también ven otros clubs, así que en cierto modo les estás regalando tus clientes a la competencia.

Las aplicaciones genéricas de gimnasios y otras soluciones similares están enfocadas más a la gestión interna (control de asistencia, rutinas, etc.) pero no ofrecen una web pública donde el club tenga su imagen de marca. Es decir, te solucionan la parte de gestión pero no la de presencia web.

Padel Around es otra aplicación muy popular en el sector, pero su enfoque principal es actuar como una red social para los jugadores. Es fantástica para hacer comunidad: permite a los usuarios buscar gente de su mismo nivel, organizar partidas abiertas y localizar pistas cercanas de forma rápida mediante geolocalización.

Webs a medida de otros clubs: Estuvimos mirando las webs de clubs deportivos de nuestra zona y la mayoría eran webs estáticas hechas con WordPress, donde la información de horarios y precios estaba desactualizada, y para reservar una pista tenías que llamar por teléfono o enviar un formulario de contacto.

Lo que nosotros queríamos era juntar todo eso en una sola cosa: una web que fuera la imagen del club (información, instalaciones, tarifas, contacto), pero que además tuviera un sistema de reservas con pagos online integrado, gestión de eventos, rutinas de gimnasio, y un panel de administración. Todo sin comisiones externas y sin depender de plataformas de terceros. Esa es la propuesta de valor de OasisClub: una solución a medida, completa, y sin costes recurrentes más allá del hosting.

5. Análisis de objetivos y metodología

Cuando presentamos la propuesta de proyecto, definimos una serie de objetivos. Aquí explicamos cómo los hemos abordado, qué decisiones tomamos, y qué dificultades nos encontramos por el camino.

OBJETIVO 1. Desarrollar una API REST con Spring Boot y Java

Álvaro se encargó de montar todo el backend con Spring Boot 4 y Java 21. Al principio valoramos usar Node.js con Express porque era lo que habíamos visto más en clase, pero al final decidimos ir con Spring Boot por varias razones: primero, porque queríamos aprender algo nuevo y que nos sirviera para el mundo laboral, ya que en las ofertas de trabajo de Murcia y alrededores veíamos mucho Java; y segundo, porque Spring Boot tiene un ecosistema muy maduro para seguridad, validación, y acceso a datos.

La verdad es que el arranque con Spring Boot fue complicado. Viniendo de JavaScript, la cantidad de configuración y anotaciones que tiene Java al principio asusta. Pero una vez que le cogimos el truco a la estructura de capas (controller → service → repository → entity), todo empezó a fluir.

La API la organizamos por módulos funcionales: auth, clients, courts, reservations, events, gym, payments, profile, schedule, newsletter y contact. Cada módulo tiene su controller, su service, sus DTOs y sus entidades. Esta estructura nos ayudó mucho a no mezclar cosas y a que cada uno pudiera trabajar en su módulo sin pisarse.

OBJETIVO 2. Implementar autenticación y autorización con JWT

Este objetivo nos dio bastante guerra. Implementamos un sistema de autenticación basado en JWT con dos roles: ADMIN y MEMBER. El admin puede gestionar todo (clientes, pistas, reservas, eventos), y el member solo puede operar sobre sus propios datos.

Lo que más nos costó fue el tema de la invalidación de tokens. Cuando un usuario cambia su contraseña, los tokens que se habían emitido antes deberían dejar de funcionar. Para solucionar esto, añadimos un campo password_changed_at en la tabla de clientes, y en el filtro JWT comparamos la fecha de emisión del token con esa fecha. Si el token se emitió antes del último cambio de contraseña, se rechaza. Este detalle nos llevó bastante tiempo de investigación pero estamos contentos de haberlo implementado porque es algo que muchos proyectos no hacen.

También implementamos un rate limiter con Bucket4j y Caffeine para proteger los endpoints sensibles: login (10 intentos cada 15 minutos por IP), recuperación de contraseña (5 cada hora), formulario de contacto (5 cada hora) y suscripción a la newsletter (5 cada hora).

Además, hay un límite global de 100 peticiones por minuto por IP que aplica a todos los endpoints, protegiendo la API contra posibles ataques de denegación de servicio. Esto lo hicimos después de leer sobre ataques de fuerza bruta y nos pareció que era importante tenerlo.

OBJETIVO 3. Diseñar y gestionar la base de datos con PostgreSQL y Flyway

Para la base de datos elegimos PostgreSQL porque era la que más nos gustaba de las que habíamos probado, y porque es la que usa Render en su plan gratuito. La gestión de

migraciones la hicimos con Flyway, que nos permite versionar los cambios del esquema de base de datos.

El esquema tiene 11 tablas: clients, courts, reservations, schedule_slots, gym_routine_days, gym_routine_exercises, password_reset_tokens, events, event_registrations, stripe_processed_events y email_failures. Lo que más nos costó fue diseñar bien la tabla de reservas, porque tenía que servir tanto para reservas normales de socios como para bloques de mantenimiento (cuando el admin cierra una pista para un evento o reparación). Al final usamos un campo status con tres valores posibles: PENDING, CONFIRMED y MAINTENANCE.

Un índice único en (court_id, reservation_date, reservation_time) nos asegura a nivel de base de datos que no se puedan hacer dos reservas en la misma pista, fecha y hora. Esto fue algo que aprendimos investigando: la validación en el código está bien, pero si dos peticiones llegan al mismo tiempo, solo la base de datos puede garantizar que no haya conflicto.

OBJETIVO 4. Desarrollar el frontend con Angular

Yo (Emilio) me encargué del frontend con Angular 21. Usamos las últimas características de Angular: componentes standalone (sin módulos), Signals para el estado reactivo, functional guards para las rutas, functional interceptors para el HTTP, y lazy loading en todas las rutas para que la app cargue rápido.

La parte que más me costó fue la gestión del estado de autenticación. Implementé un AuthStore basado en Signals que guarda la sesión en localStorage y la expone como señales reactivas. Cuando el token caduca o el backend devuelve un 401 o 403, un interceptor de errores cierra la sesión automáticamente y redirige al login. Esto parece simple pero conseguir que todo funcionara bien junto (interceptors, guards, store, y navegación) me llevó bastante tiempo.

Para el diseño utilicé TailwindCSS 4, que al principio me costó un poco porque venía de usar CSS normal, pero una vez que le coges el ritmo es muy rápido para maquetar.

También configuré un sistema de optimización de imágenes con Sharp para que las fotos de la web no pesaran demasiado.

OBJETIVO 5. Integrar un sistema de pagos con Stripe

La integración con Stripe fue probablemente lo más complejo de todo el proyecto. El flujo funciona así: cuando un socio quiere reservar una pista, se crea una reserva en estado PENDING y se genera una sesión de checkout de Stripe. El socio paga en la pasarela de Stripe, y cuando el pago se confirma, Stripe nos envía un webhook que cambia la reserva a CONFIRMED. Si el pago falla o expira, otro webhook libera el slot.

Lo que más nos costó fue hacer el sistema idempotente. Stripe puede enviar el mismo webhook varias veces, y si no controlas eso puedes acabar confirmando la misma reserva dos veces o cosas peores. Para solucionarlo creamos una tabla stripe_processed_events donde guardamos el ID de cada evento de Stripe que ya hemos procesado, y si nos llega uno repetido simplemente lo ignoramos.

Otra cosa complicada fue el tema de las URLs de retorno. Dependiendo de si el usuario está en localhost, en el deploy de Vercel, o en cualquier otro origen, las URLs de éxito y cancelación del checkout tienen que cambiar. Implementamos una lógica que detecta el origen de la petición y usa la URL correcta.

Los precios están definidos en el código: 100€ para fútbol y 50€ para pádel. Sabemos que lo ideal sería tener una tabla de precios configurable desde el panel de administración, pero por tiempo no nos dio a implementarlo.

OBJETIVO 6. Implementar un sistema de correos electrónicos

Configuramos el envío de correos con Gmail SMTP a través de Spring Boot Mail. Se envían correos automáticos para: confirmación de registro, recuperación de contraseña, confirmación de reserva, cancelación de reserva, inscripción a eventos, y baja de eventos. Los templates de los correos los hicimos directamente en HTML dentro del código Java, que es algo que sabemos que no es lo ideal (el archivo EmailService tiene bastante código), pero para el alcance del proyecto funcionaba bien. Para producción, lo suyo sería usar un motor de templates como Thymeleaf.

Un detalle que nos parece importante mencionar es que implementamos un sistema de registro de fallos de correo. Si un email no se puede enviar, se guarda en la tabla `email_failures` con el destinatario, el tipo de email, el asunto y el error. Esto nos permite saber qué correos han fallado y por qué, y en un futuro podríamos implementar un sistema de reintentos automáticos.

OBJETIVO 7. Desplegar la aplicación en producción

Para el despliegue, el backend lo pusimos en Render (plan gratuito) y el frontend en Vercel. Docker nos ayudó mucho durante el desarrollo porque teníamos un `docker-compose.dev.yml` que levantaba PostgreSQL, el backend con hot-reload, y el frontend con proxy al backend, todo con un solo comando.

Render tiene un problema con el plan gratuito: el servicio se apaga después de un rato de inactividad y la primera petición tarda mucho en responder (cold start). Para mitigar esto, implementamos un health endpoint que se puede llamar periódicamente para mantener el servicio activo.

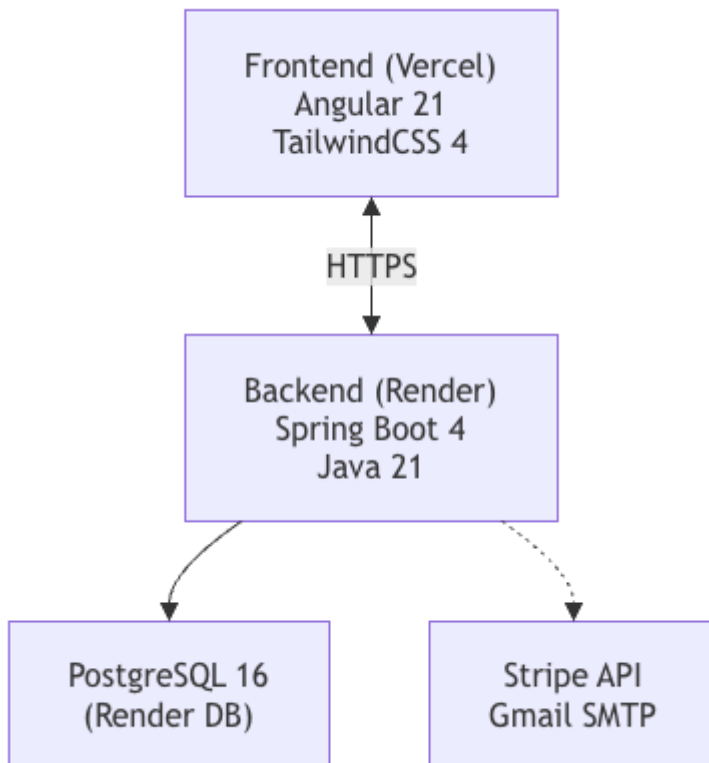
La configuración de CORS nos dio bastantes dolores de cabeza al principio. Entre localhost con diferentes puertos, el dominio de Vercel, y el dominio de Render, había que tener todas las URLs bien configuradas. Al final lo resolvimos con variables de entorno y patterns de CORS que aceptan subdominios.

Metodología de trabajo

Para organizarnos usamos GitHub como repositorio central, trabajando en la rama main. La comunicación era diaria por Discord, y nos juntábamos presencialmente para las integraciones más importantes. No seguimos una metodología ágil formal tipo Scrum, sino que teníamos una lista de funcionalidades pendientes e íbamos cogiendo tareas según la prioridad.

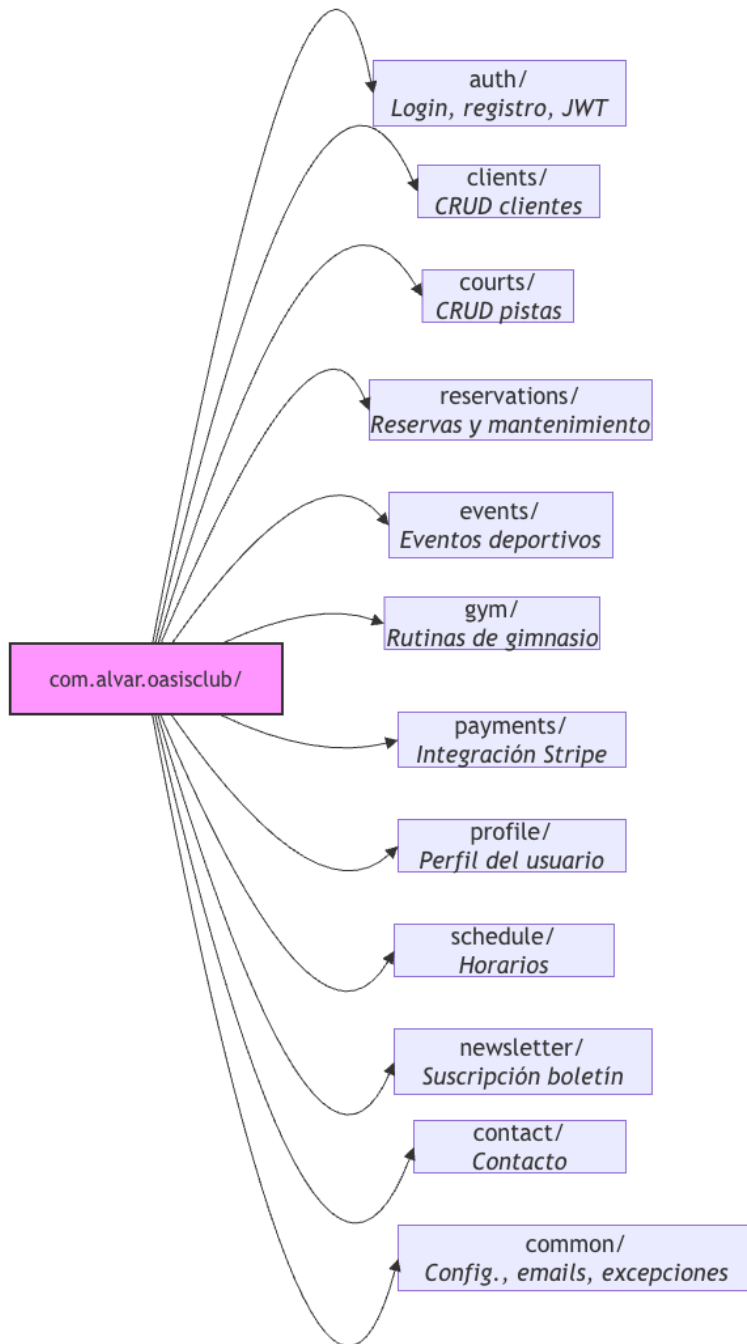
6. Diseño y resolución del trabajo realizado

La aplicación sigue una arquitectura cliente-servidor clásica con separación clara entre frontend y backend. Toda la comunicación entre ambos se hace a través de una API REST bajo el prefijo `/api/v1/`.



6.2 Estructura del backend

El backend está organizado por módulos funcionales. Cada módulo tiene su propia carpeta con subcarpetas para controllers, services, DTOs, entities, mappers, repositories y excepciones. Esta organización la sacamos de investigar sobre "package by feature" versus "package by layer", y nos pareció más limpia porque cada módulo es independiente.



Algo de lo que estamos orgullosos es del `GlobalExceptionHandler`. Es un controlador de errores centralizado que captura todas las excepciones de la aplicación y las convierte en respuestas JSON con mensajes en español, claros para el usuario. Por ejemplo, si un campo de un formulario no es válido, en vez de devolver el típico mensaje en inglés de Java, devuelve algo como "El correo electrónico es obligatorio" o "La contraseña debe tener al menos 8 caracteres". Esto nos llevó bastante tiempo pero hace que la experiencia del usuario sea mucho mejor.

java

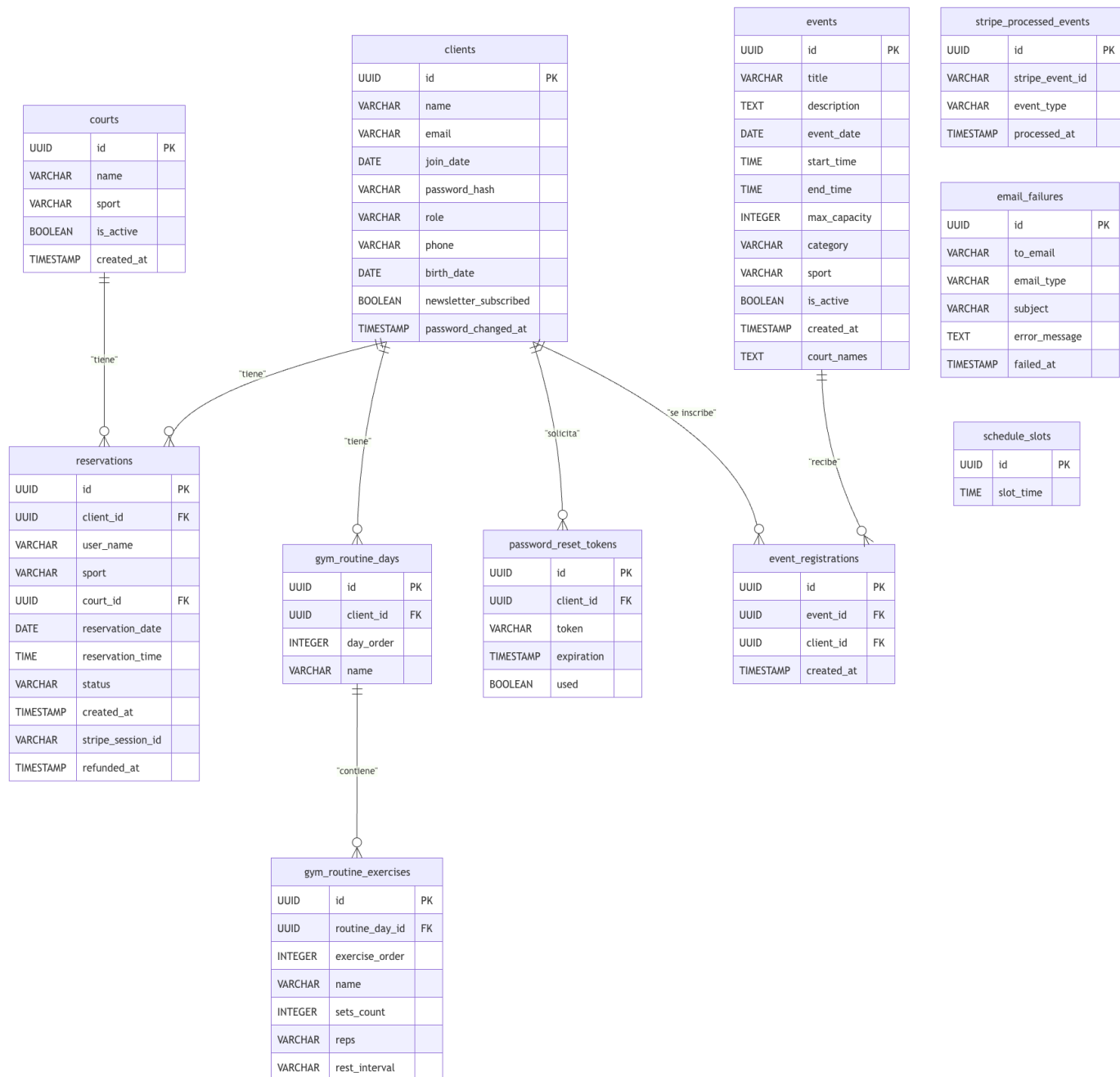
// Ejemplo del `GlobalExceptionHandler` - traducción de errores de validación



```
private String formatFieldError(FieldError error) {
    String friendly = friendlyFieldName(error.getField());
    String code = error.getCode() != null ? error.getCode().toLowerCase() : "";
    return switch (code) {
        case "notblank", "notempty" -> friendly + " es obligatorio";
        case "email" -> friendly + " debe ser un correo electrónico válido";
        case "size" -> {
            // ... lógica para min/max de caracteres
            yield friendly + " debe tener entre " + mn + " y " + mx + " caracteres";
        }
        default -> friendly + ": " + translateConstraintMessage(error.getDefaultMessage());
    };
}
```

6.3 Esquema de base de datos

El diseño de la base de datos fue algo que trabajamos juntos y que nos costó varias iteraciones. Aquí mostramos las tablas principales y sus relaciones:



Una decisión importante fue usar UUIDs en vez de IDs numéricos autoincrement. Lo hicimos porque en una API REST, exponer IDs numéricos hace que sea fácil adivinar los IDs de otros recursos (si mi reserva es la 45, la de otro será la 44 o la 46). Con UUIDs eso no pasa. Al principio pensamos que iba a complicar las cosas, pero con JPA y GenerationType.UUID fue bastante sencillo.

6.4 Seguridad

El sistema de seguridad tiene varias capas:

1. **Rate Limiting:** Un filtro que limita las peticiones por IP. Por ejemplo, el login permite 10 intentos cada 15 minutos. Si te pasas, te devuelve un error 429 con un mensaje

indicando cuánto tiempo tienes que esperar. Usamos Bucket4j para los buckets de tokens y Caffeine como caché para guardarlos en memoria.

2. JWT Filter: Extrae el token JWT del header Authorization, lo valida, comprueba que el rol del usuario no haya cambiado, y verifica que la contraseña no se haya cambiado después de la emisión del token.

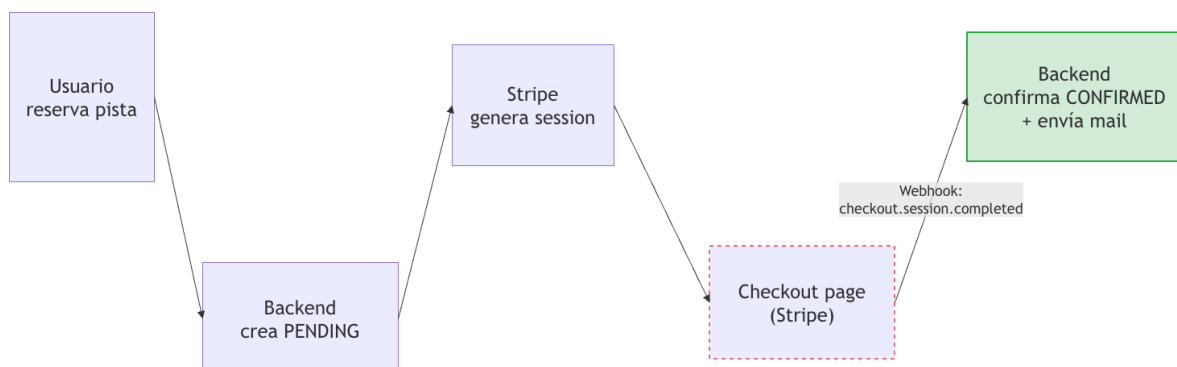
```
// JwtService.java - Generación del token en el Backend
public String generateToken(UUID clientId, String email, String role) {
    Date now = new Date();
    Date expiration = new Date(now.getTime() + expirationMs);

    return Jwts.builder()
        .subject(email)
        .claim("clientId", clientId.toString())
        .claim("role", role)
        .issuedAt(now)
        .expiration(expiration)
        .signWith(getSigningKey())
        .compact();
}
```

3. Security Config: Define qué endpoints son públicos (login, registro, Swagger) y cuáles requieren autenticación.
4. Access Control Service: Métodos como requireAdmin(), requireUser(), requireAdminOrOwner() que se usan en los controllers para verificar permisos.

6.5 Sistema de pagos con Stripe

Este fue el bloque que más investigación requirió. El flujo completo es:



El mayor desafío fue la idempotencia. Stripe puede enviar un mismo evento varias veces (por ejemplo, si nuestro servidor no responde a tiempo, Stripe reintenta). Si no controlamos eso, podríamos confirmar una reserva dos veces. La solución fue crear la tabla `stripe_processed_events` donde guardamos el ID de cada evento procesado:

java

```
// PaymentWebhookProcessor.java - Control de idempotencia
if (processedEventRepository.existsByStripeEventId(eventId)) {
    log.info("[WEBHOOK] Event {} already processed, skipping", eventId);
    return;
}
// ... procesar el evento
processedEventRepository.save(new StripeProcessedEventEntity(eventId, eventType));

};
}
```

6.6 Frontend Angular

El frontend está hecho con Angular 21 usando las últimas características del framework. Todos los componentes son standalone (sin NgModule), lo cual simplifica mucho la configuración y permite hacer lazy loading de cada componente individualmente. La gestión del estado de autenticación la hice con un servicio basado en Signals: typescript

```
// auth.store.ts - Estado reactivo con Signals
@Injectable({providedIn: 'root'})
export class AuthStore {
    private readonly _session = signal<AuthSession | null>(this.loadFromStorage());
    readonly session = this._session.asReadonly();
    readonly isLoggedIn = computed(() => this._session() !== null);
    readonly isAdmin = computed(() => this._session()?.role === 'ADMIN');
}
```

Esto es mucho más limpio que usar BehaviorSubject de RxJS, que era lo que habíamos visto en clase. Los Signals se actualizan automáticamente, los componentes que dependen de ellos se re-renderizan solos, y no tienes que preocuparte por desuscribirte ni por memory leaks.

Las rutas están protegidas con functional guards:

typescript


```
// Guards funcionales para proteger rutas
export const adminOnlyGuard: CanActivateFn = () => {
  const authStore = inject(AuthStore);
  const router = inject(Router);
  if (!authStore.isLoggedIn()) return router.createUrlTree(['/login']);
  if (authStore.isAdmin()) return true;
  return router.createUrlTree(['/']);
};
donly isAdmin = computed(() => this._session()?.role === 'ADMIN');
}
```

Y los interceptores HTTP son también funcionales. Uno inyecta el token JWT en cada petición, y otro captura los errores de la API y los muestra al usuario, cerrando sesión automáticamente si el token ha expirado:

typescript

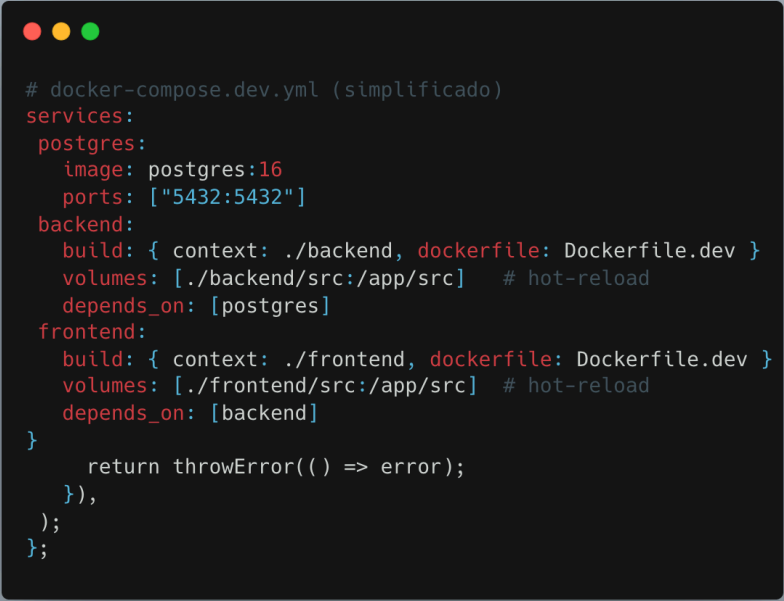
```
// api-error.interceptor.ts
export const apiErrorInterceptor: HttpInterceptorFn = (request, next) => {
  return next(request).pipe(
    catchError((error: HttpResponse) => {
      if (error.status === 0) {
        errorStore.set('No se pudo conectar con el backend.');
```

```
      } else if ((error.status === 401 || error.status === 403) && authStore.isLoggedIn()) {
        authStore.logout();
        router.navigate(['/login']);
      }
      return throwError(() => error);
    })
  );
};
```

6.7 Entorno de desarrollo con Docker

Para el desarrollo local montamos un docker-compose.dev.yml que levanta tres contenedores: PostgreSQL, el backend con hot-reload (Spring DevTools), y el frontend con proxy al backend. Esto nos permitía tener el entorno completo funcionando con un solo docker compose up, sin tener que instalar ni configurar nada manualmente.

yaml



```
# docker-compose.dev.yml (simplificado)
services:
  postgres:
    image: postgres:16
    ports: ["5432:5432"]
  backend:
    build: { context: ./backend, dockerfile: Dockerfile.dev }
    volumes: [./backend/src:/app/src] # hot-reload
    depends_on: [postgres]
  frontend:
    build: { context: ./frontend, dockerfile: Dockerfile.dev }
    volumes: [./frontend/src:/app/src] # hot-reload
    depends_on: [backend]
}

return throwError(() => error);
}),
);
};
```

6.8 Gestión de eventos

El módulo de eventos fue algo que añadimos más adelante en el proyecto, pero estamos bastante contentos del resultado. Un admin puede crear eventos deportivos (torneos, clases colectivas, etc.) y el sistema automáticamente bloquea las pistas asociadas para que nadie pueda reservarlas durante el horario del evento. Cuando el evento se elimina o pasa su fecha, los bloques de mantenimiento se liberan automáticamente.

Los socios pueden inscribirse y desinscribirse de los eventos desde la web, reciben un correo de confirmación, y hay un control de aforo máximo con un bloqueo pesimista en la base de datos (SELECT ... FOR UPDATE) para evitar que se inscriba más gente de la capacidad permitida aunque hagan click al mismo tiempo.

7. Presupuesto

El cálculo de horas lo hemos hecho en base al tiempo real que hemos dedicado al proyecto, incluyendo investigación, desarrollo, pruebas, y despliegue.

Horas por fase

Fase	Emilio (Frontend)	Álvaro (Backend)	Horas totales
Investigación y análisis	10h	10h	10h
Diseño y maquetación	20h	5h	25h
Desarrollo frontend	80h	—	80h
Desarrollo backend	—	90h	90h
Integración y pruebas	10h	15h	25h
Integración Stripe	—	5h	5h
Sistema de emails y diseño	5h	15h	20h
Despliegue	—	10h	10h
Documentación	10h	10h	20h
Total	135h	160h	285h

Presupuesto económico

Si trasladamos las horas a un presupuesto profesional, tomando como referencia un precio por hora de un desarrollador web junior (15-20€/h):

Concepto	Horas	Precio/hora	Total
Desarrollo frontend	80h	10€	800€
Desarrollo backend	90h	10€	900€
Investigación y diseño	45h	10€	450€
Integración, pruebas y deploy	35h	10€	350€
Subtotal desarrollo	250h	—	2500€

Concepto	Coste mensual	Meses	Total
Hosting Render (plan gratuito)	0€	1	12€
Hosting Vercel (plan gratuito)	0€	6	0€
Dominio (si se compra)	12€/año	—	12€
Stripe (comisión por transacción)	1,4% + 0,25€	—	Variable
Total infraestructura	—	—	~24€

--	--

Presupuesto total estimado	~2.524€
----------------------------	---------

Nota: Los costes de infraestructura son mínimos gracias al uso de planes gratuitos de Render y Vercel. En un entorno de producción real, habría que considerar planes de pago para evitar las limitaciones del plan gratuito (cold starts, límites de uso, etc.).

8. Conclusiones y vías futuras

Conclusiones técnicas

Estamos bastante satisfechos con el resultado del proyecto. Hemos conseguido hacer una aplicación web completa y funcional, con un frontend moderno, un backend robusto, pagos reales con Stripe, y un sistema de seguridad que va bastante más allá de lo que se pide normalmente en un TFG.

Sin embargo, somos conscientes de que hay cosas que se podrían mejorar. El EmailService del backend tiene todos los templates HTML metidos directamente en el código Java, lo cual hace que sea un archivo enorme y difícil de mantener. Lo ideal sería usar un motor de plantillas como Thymeleaf para externalizar esos templates. No nos dio tiempo a hacerlo, pero sabemos que es algo que habría que cambiar en un proyecto real.

Otro punto que nos quedó pendiente es el tema de los tests en el frontend. El backend tiene tests unitarios para todos los servicios, pero en el frontend no llegamos a implementar tests.

Esto es algo que nos habría gustado hacer, pero al final el tiempo se nos echó encima.

El rate limiting funciona bien en local, pero en producción (Render), al estar detrás de un proxy, la IP que recibimos puede ser siempre la del proxy y no la del usuario real. Habría que parsear el header X-Forwarded-For para solucionarlo, pero no lo implementamos por falta de tiempo y porque requiere tener cuidado con la seguridad (ese header se puede falsificar).

Los precios de las reservas están hardcodedos en el código. En un proyecto real, deberían estar en una tabla de la base de datos y ser configurables desde el panel de administración.

Conclusiones personales

A nivel personal, este proyecto nos ha enseñado mucho más de lo que esperábamos.

Aprender Spring Boot viniendo de JavaScript ha sido un reto, pero nos ha abierto la puerta a un mundo que tiene mucha demanda laboral. Angular con Signals y standalone components es otro paradigma comparado con lo que vimos en clase, y estamos contentos de haber apostado por la versión más nueva aunque eso significara menos recursos y tutoriales disponibles.

Una de las cosas que más nos ha enseñado este proyecto es a trabajar en equipo de verdad. No es lo mismo hacer un trabajo en grupo donde cada uno hace su parte y se junta al final, que trabajar en un proyecto donde el trabajo de uno depende del trabajo del otro.

Tuvimos que aprender a definir contratos (qué endpoints, qué DTOs, qué códigos de error), a comunicarnos cuando algo cambiaba, y a solucionar conflictos de integración. También hemos aprendido que el desarrollo de software no es solo programar. El tiempo que dedicamos a investigar, a descartar opciones, a diseñar la base de datos, a configurar el despliegue, y a depurar errores en producción ha sido igual o mayor que el tiempo de programación pura. Y eso es algo que en clase no se ve tanto.

Vías futuras

Si tuviéramos que seguir desarrollando OasisClub, estas serían las siguientes funcionalidades que implementaríamos:

- Panel de precios configurable: Que el admin pueda cambiar los precios de las reservas sin tocar código.
- Notificaciones push: Para avisar a los socios de nuevos eventos o cambios en sus reservas.
- App móvil con PWA: Convertir la web en una Progressive Web App para que se pueda instalar en el móvil.
- Sistema de abonos/suscripciones: Planes mensuales con Stripe Subscriptions en vez de pagos puntuales por reserva.
- Estadísticas y dashboard: Gráficas de ocupación, ingresos, eventos más populares, etc.
- Tests end-to-end con Playwright: Para automatizar las pruebas del frontend completo.
- Internacionalización (i18n): Soporte para múltiples idiomas.

9. Bibliografía/Webgrafía

- Angular. (s.f.). Angular documentation. Recuperado de <https://angular.dev/>
- Baeldung. (s.f.). Spring Boot tutorials. Recuperado de <https://www.baeldung.com/>
- Docker Inc. (s.f.). Docker documentation. Recuperado de <https://docs.docker.com/>
- Flyway. (s.f.). Flyway documentation. Recuperado de <https://flywaydb.org/documentation/>
- JSON Web Tokens. (s.f.). Introduction to JSON Web Tokens. Recuperado de <https://jwt.io/introduction>
- Mozilla Developer Network. (s.f.). MDN Web Docs. Recuperado de <https://developer.mozilla.org/>
- Playtomic. (s.f.). Playtomic — Reserva pistas de pádel y tenis. Recuperado de <https://playtomic.io/>
- PostgreSQL Global Development Group. (s.f.). PostgreSQL documentation. Recuperado de <https://www.postgresql.org/docs/>
- Render. (s.f.). Render documentation. Recuperado de <https://docs.render.com/>
- Spring. (s.f.). Spring Boot reference documentation. Recuperado de <https://docs.spring.io/spring-boot/>
- Stripe. (s.f.). Stripe API documentation. Recuperado de <https://docs.stripe.com/>
- Stripe. (s.f.). Stripe Checkout integration guide. Recuperado de <https://docs.stripe.com/checkout>
- Google. "Gemini". 2026. <https://gemini.google.com>.
- Stripe. (s.f.). Handling webhook events. Recuperado de <https://docs.stripe.com/webhooks>
- Syltek. (s.f.). Syltek — Software de gestión de clubs deportivos. Recuperado de <https://www.syltek.com/>
- TailwindCSS. (s.f.). TailwindCSS documentation. Recuperado de <https://tailwindcss.com/docs>
- Vercel. (s.f.). Vercel documentation. Recuperado de <https://vercel.com/docs>
- VMware Tanzu. (s.f.). Spring Security reference. Recuperado de <https://docs.spring.io/spring-security/>
- Bucket4j. (s.f.). Bucket4j — Java rate limiting library. Recuperado de <https://github.com/bucket4j/bucket4j>
- MapStruct. (s.f.). MapStruct reference guide. Recuperado de <https://mapstruct.org/documentation/reference-guide/>
- Project Lombok. (s.f.). Project Lombok features. Recuperado de <https://projectlombok.org/features/>